

Функции в Python

доц. д-р Гергана Спасова

Кат. „Компютърни науки и технологии“

Вградени функции

- `abs()`
- `min()`
- `max()`
- `divmod()`
- `pow()`
- `len()`

Вградени функции за помощна информация

- `Dir()` е вградена функция, която връща списък с имената на всички функции, класове и променливи, дефинирани в модула параметър на функцията

Пример:

```
>>> dir(math)
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',  
'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp',  
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf',  
'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf',  
'nan', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
```

Вградени функции за помощна информация

- Функция `help()` – параметър символен низ, който може да бъде име на модул, функция, клас, метод, ключова дума или тема, за която търсим помощ

Пример:

```
help(math.gcd)
```

Help on built-in function gcd in module math:

```
gcd(...)
```

```
gcd(x, y) -> int
```

```
greatest common divisor of x and y
```

Дефиниране на потребителски функции

```
def function_name(parameter_1, parameter_2, ..., parameter_n):  
    statement(s)
```

- Много често първият оператор в тялото на функцията е коментар, който се използва при генериране на документация от online инструменти:

```
"""This is single line docstring """
```

OR

```
"""This is  
multiline  
docstring """
```

Извикване на функция

- `function_name(argument_1, argument_2,...,argument_n)`
- Преди изпълнение на програма Python интерпретаторът автоматично инициализира някои специални променливи. Такава специална променлива е `__name__`.
- Когато стартираме файл с Python скрипт като главна програма, интерпретаторът присвоява на променливата `__name__` стойност "main".
- Когато файл с Python скрипт се импортира в друг модул, променливата `__name__` има стойност името на модула, в който е импортиран скриптът.

Аргументи на функции

Пример 1:

```
def multiply(a, b=2):  
    return a * b
```

Изход:

```
multiply(5)  
multiply(5, 10)
```

Пример 2:

```
def is_pythagorean(a=2, b=3, c=4):  
    return a * a + b * b == c * c
```

Изход:

```
is_pythagorean(b=5)  
is_pythagorean(1, c=3)
```

Променлив брой аргументи

```
def varfunc(some_arg, *args, **kwargs):
```

```
varfunc('hello', 1, 2, 3, name='Bob', age=12)
```

Напишете функция, която принтира думата „пролет“. Написаната функция се извиква от друга функция пет пъти. Какъв ще бъде резултата?

Глобална променлива `__main__`

```
# Python program to execute main directly
```

```
print ("Always executed")
```

```
if __name__ == "__main__":
```

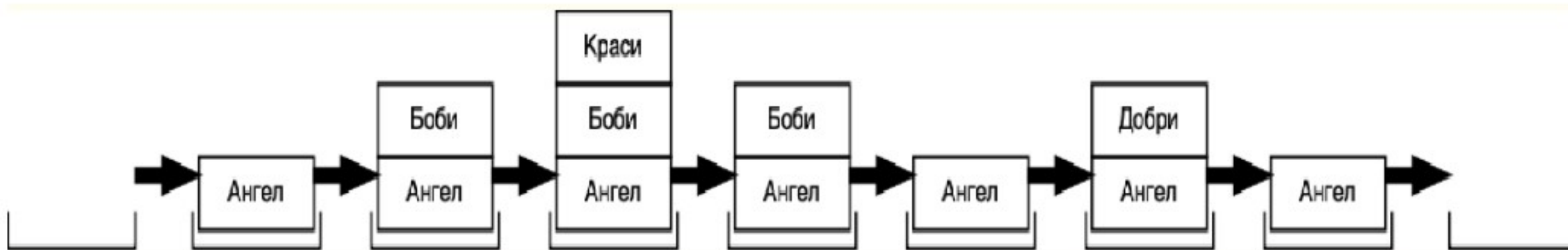
```
    print ("Executed when invoked directly")
```

```
else:
```

```
    print ("Executed when imported")
```

- https://www.geeksforgeeks.org/what-does-the-if-__name__-__main__-do/
- <https://www.freecodecamp.org/news/if-name-main-python-example/>

Стек на извикванията (call stack)



Пример

Програма за намиране лице на трапец по формула $\text{Area} = (1/2) * (a + b) * h$, където **a** и **b** са основи, а **h** е височината на трапеца

```
def area_trapezium(a, b, h):  
    area = 0.5 * (a + b) * h  
    print(f"Area of a Trapezium is {area}")  
  
def main():  
    area_trapezium(10, 15, 20)  
  
if __name__ == "__main__":  
    main()
```

Изход:

Area of a Trapezium is 250.0

Пример

- Програма, която демонстрира използване на променлива с едно и също име при дефиниране и извикване на функция

```
god_name = input("Who is the God of Seas according to Greek Mythology?")
def greek_mythology(god_name):
    print(f"The God of seas according to Greek Mythology is {god_name} ")
def main():
    greek_mythology(god_name)
if __name__ == "__main__":
    main()
```

Изход

Who is the God of Seas according to Greek Mythology? Poseidon
The God of seas according to Greek Mythology is Poseidon

Оператор `return` и `void` функции

- Операторът `return` се използва за връщане на резултат от изпълнение на функция; не е задължителен в тялото на функцията
`return [expression_list]`
- Операторът `return` връща стойност, изчислена в резултат от изразите в `expression_list`
- Ако тялото на функцията не съдържа оператор `return`, тя връща стойност `None`
- `Return` приключва изпълнението на функцията
- Функцията в Python може да връща единични стойности, разделение със запетая; може да връща също списък или `tuple`

Пример

Програма, която демонстрира връщане на множество стойности в дефиниция на функция

```
def world_war():  
    alliance_world_war = input("Which alliance won World War 2?")  
    world_war_end_year = input("When did World War 2 end?")  
    return alliance_world_war, world_war_end_year  
  
def main():  
    alliance, war_end_year = world_war()  
    print(f"The war was won by {alliance} and the war ended in {war_end_year}")  
  
if __name__ == "__main__":  
    main()
```

Изход:

Which alliance won World War 2? Allies

When did World War 2 end? 1945

The war was won by Allies and the war ended in 1945

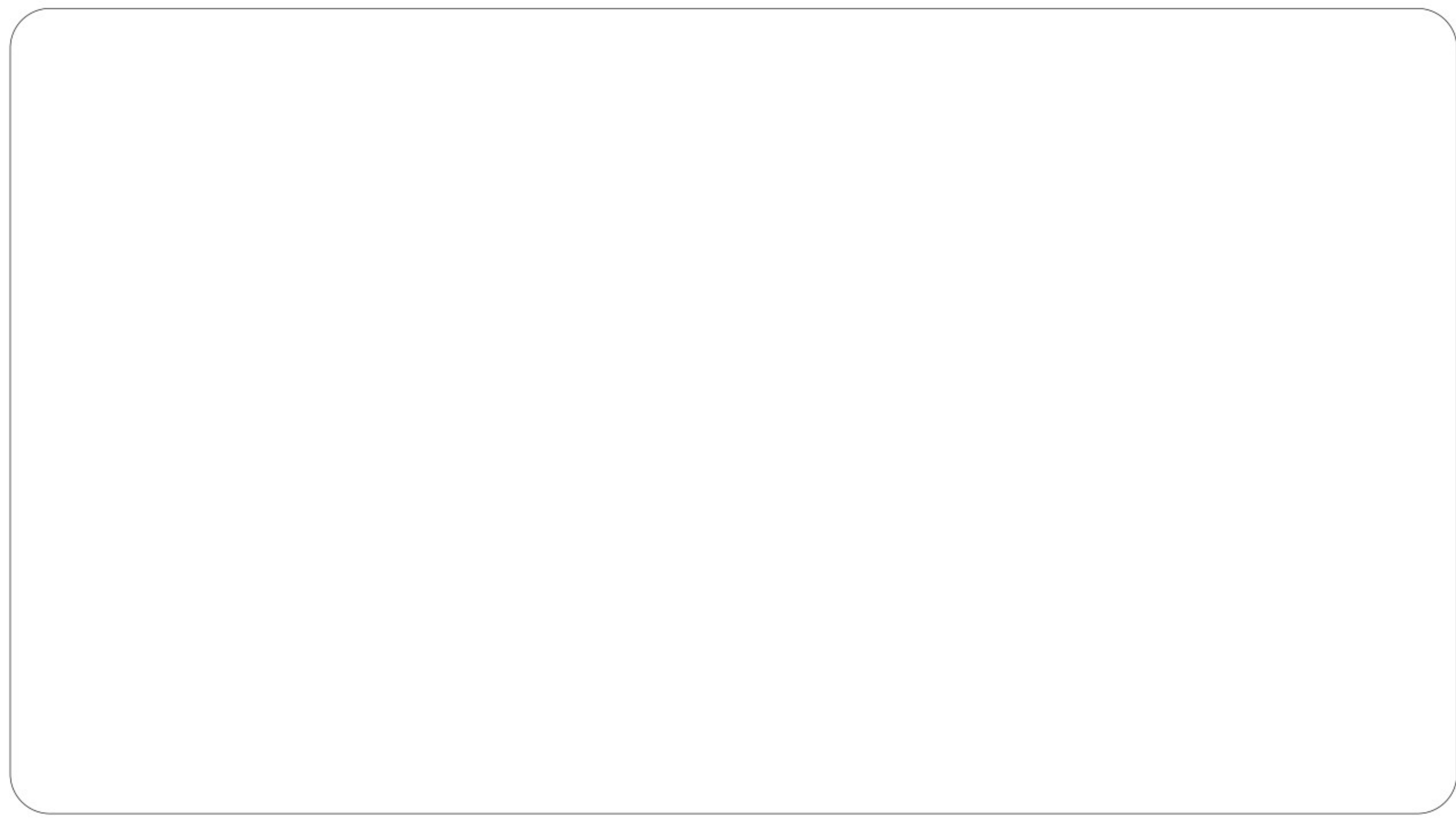
Област на видимост на променливи (scope)

- Глобални – област на видимост в цялата програма
- Локални – област на видимост само във функция

Обхватите имат значение по няколко причини :

- Кодът в глобалния обхват (извън всички функции) не може да използва нито една от локалните променливи.
- Но код в локален обхват може да достъпи глобалните променливи.
- Код в локалния обхват на една функция не може да използва променливи от друг локален обхват.

Може да има различни променливи с еднакви имена, ако са в различни обхвати. Тоест може да има локална променлива с име `sum` и глобална променлива с име `sum`.



Пример

```
1. test_variable = 5
2. def outer_function():
3.     test_variable = 60
4.     def inner_function():
5.         test_variable = 100
6.         print(f"Local variable value of {test_variable} having local scope to inner function
is displayed")
7.     inner_function()
8.     print(f"Local variable value of {test_variable} having local scope to outer function
is displayed ")
9. outer_function()
10. print(f"Global variable value of {test_variable} is displayed ")
```

Исход:

Local variable value of 100 having local scope to inner function is displayed

Local variable value of 60 having local scope to outer function is displayed

Global variable value of 5 is displayed

Ключова дума `nonlocal`

- `Nonlocal` се използва при работа с променливи във вложена функция, когато променливата не трябва да бъде локална за вложената функция.

Пример:

```
def myfunc1():  
    x = "John"  
    def myfunc2():  
        nonlocal x  
        x = "hello"  
    myfunc2()  
    return x  
  
print(myfunc1())
```

Параметри по подразбиране

```
def work_area(prompt, domain="Data Analytics"):
    print(f" {prompt} {domain} ")
def main():
    work_area("Sam works in")
    work_area("Alice has interest in", "Internet of Things")
if __name__ == "__main__":
    main()
```

Изход:

Sam works in Data Analytics

Alice has interest in Internet of Things

Аргументи ключови думи

- В извикващата функция могат да бъдат задавани имена на параметри и съответни стойности във формат:

key = value

```
def parrot(voltage, state='a stiff', action='vroom',
type='Norwegian Blue'):
    print(f"This parrot wouldn't {action}, if you put {voltage},
volts through it.")
    print(f"Lovely plumage, the {type}")
    print(f"It's {state} !!!")
parrot(1000)
parrot(voltage=1000)
parrot(voltage=1000000, action='VOOOOOOM')
parrot('a thousand', state='pushing up the daisies')
```

Исход:

This parrot wouldn't vroom, if you put 1000, volts through it.
Lovely plumage, the Norwegian Blue
It's a stiff !!!

This parrot wouldn't vroom, if you put 1000, volts through it.
Lovely plumage, the Norwegian Blue
It's a stiff !!!

This parrot wouldn't VOOOOOOM, if you put 1000000, volts
through it.
Lovely plumage, the Norwegian Blue
It's a stiff !!!

This parrot wouldn't vroom, if you put a thousand, volts through it.
Lovely plumage, the Norwegian Blue
It's pushing up the daisies !!!

Следните извиквания в предния пример са неправилни:

- `parrot()` # липсва аргумент
- `parrot(voltage=5.0, 'dead')` # след ключова дума и стойност има
аргумент, който не е с ключова дума
- `parrot(110, voltage=220)` # две различни стойности за един и същ
аргумент
- `parrot(actor='John Cleese')` # невалиден аргумент ключова дума

`*args` и `**kwargs`

- Използват се, когато не е известен броят на параметрите за извикване на функцията
- `**kwargs` позволява предаване на променлив брой параметри
- `*args` съдържа списък с параметри, които се предават на извикваната функция

Примерна програма с аргументи *args и *kwargs

```
def cheese_shop(kind, *args, **kwargs):  
    print(f"Do you have any {kind} ?")  
    print(f"I'm sorry, we're all out of {kind} ")  
    for arg in args:  
        print(arg)  
    print("-" * 40)  
    for kw in kwargs:  
        print(kw, ":", kwargs[kw])  
  
def main():  
    cheese_shop("Limburger", "It's very runny, sir.",  
                "It's really very, VERY runny, sir.",  
                shop_keeper="Michael Palin",  
                client="John Cleese",  
                sketch="Cheese Shop Sketch")  
  
if __name__ == "__main__":  
    main()
```

ИЗХОД:

Do you have any Limburger ?

I'm sorry, we're all out of Limburger

It's very runny, sir.

It's really very, VERY runny, sir.

shop_keeper : Michael Palin

client : John Cleese

sketch : Cheese Shop Sketch

Параметри от командния ред

- Всички параметри на командния ред, с които се извиква Python скрипт, се съдържат в

```
import sys
```

```
def main():
```

```
    print(f"sys.argv prints all the arguments at the command line including file name {sys.argv}")
```

```
    print(f"len(sys.argv) prints the total number of command line arguments including file name {len(sys.argv)}")
```

```
    print("You can use for loop to traverse through sys.argv")
```

```
    for arg in sys.argv:
```

```
        print(arg)
```

```
if __name__ == "__main__":
```

```
    main()
```

Исход

```
C:\Introduction_To_Python_Programming\Chapter_4>python Program_4.12.py arg_1  
arg_2 arg_3
```

sys.argv prints all the arguments at the command line including file name ['Program_4.12.py', 'arg_1', 'arg_2', 'arg_3']

len(sys.argv) prints the total number of command line arguments including file name 4

You can use for loop to traverse through sys.argv

```
Program_4.12.py
```

```
arg_1
```

```
arg_2
```

```
arg_3
```

Още за дефиниране и извикване на функции

- <https://www.programiz.com/python-programming/function-argument>

Въпроси?